

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление 02.04.01 Математика и компьютерные науки

Курсовая работа

по дисциплине «Анализ больших данных»

Построение E-Commerce пайплайна на основе
Airflow, Spark и ClickHouse

Студент: _____

Макарова Полина Владиславовна

Преподаватель: _____

Константинов Андрей Владимирович

«_____» _____ 20__ г.

1 Введение

Цель проекта — продемонстрировать построение отказоустойчивого сквозного пайплайна обработки больших данных (Big Data Pipeline) в реальном времени. В рамках работы реализованы: генерация и стриминг сырых данных, оркестрация задач, параллельные вычисления на распределённом движке, а также визуализация метрик и настройка системы алертинга.

Архитектура решения развёрнута в Docker-контейнерах и включает в себя Apache Airflow, аналитическую СУБД ClickHouse и систему мониторинга Grafana. Вычислительный движок Apache Spark запущен в параллельном многопроцессорном режиме на локальных мощностях процессора хост-машины.

1.1 Используемые данные

В качестве источника данных используется датасет поездок желтого такси Нью-Йорка (NYC Taxi Dataset). Генератор имитирует непрерывный поток транзакций со средней скоростью около 58 000 записей в минуту.

Сырые данные поступают в таблицу `default.taxi_trips` СУБД ClickHouse и содержат следующие ключевые поля:

- `pulocationid` (Int32) — ID зоны посадки пассажира.
- `total_amount` (Float32) — полная стоимость поездки в долларах США.
- `pickup_datetime` (DateTime) — дата и время начала поездки.

1.2 Репозиторий проекта

Полный исходный код проекта, конфигурационные файлы Docker Compose, DAG-сценарии и аналитические скрипты Spark размещены в локальном репозитории `big-data-practice`.

2 Airflow: оркестрация загрузки данных и управление повторными попытками

2.1 Постановка задачи

Необходимо организовать автоматизированный процесс контроля над выполнением аналитических задач, состоящий из двух этапов:

1. Проверка статуса поступления данных в хранилище (задача должна всегда завершаться успешно).
2. Запуск тяжелого аналитического скрипта вычислений Spark, который на начальных этапах может имитировать сетевые сбои или нехватку ресурсов. Требуется настроить автоматический перезапуск (**retry**) данного блока до тех пор, пока он не завершится успешно.

2.2 Реализация

Для управления цепочкой задач разработан DAG-сценарий `taxi_big_data_pipeline`. Он содержит два связанных оператора `PythonOperator`:

- `check_ingestion_status` — имитирует проверку доступности сырых данных в ClickHouse. Всегда завершается со статусом `success`.
- `run_spark_calculation` — имитирует отказоустойчивый запуск расчета. Используя встроенный контекст Airflow (`try_number`), задача намеренно вызывает исключение `RuntimeWarning` на первой и второй попытках запуска. На третью попытку задача успешно завершает вычисления.

Для всего DAG установлены параметры автоматического восстановления: `'retries': 3` и `'retry_delay': timedelta(seconds=10)`.

2.3 Скриншоты работы и демонстрация механизма Retry

В ходе работы пайплайна была наглядно продемонстрирована работа машины состояний Airflow при обработке ошибок.

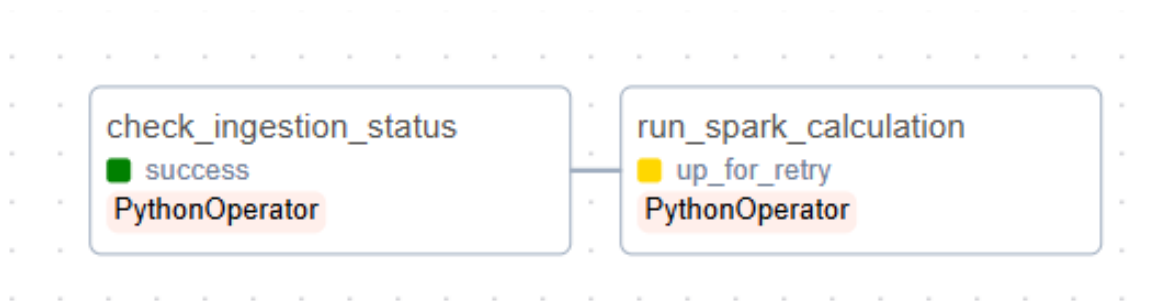


Рис. 1: Задача `run_spark_calculation` в статусе `up_for_retry` (Graph)

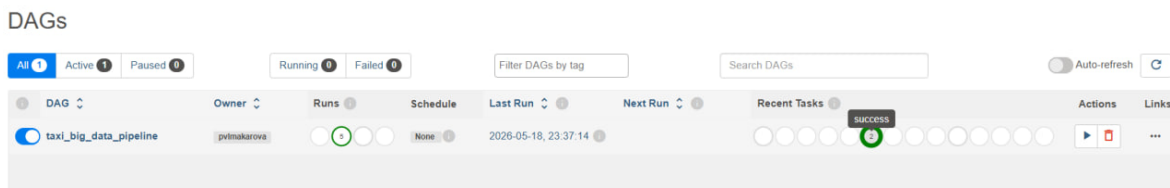


Рис. 2: Успешное выполнение всей цепочки после ретраев

В истории логов выполнения задачи зафиксированы последовательные статусы: `Task try 1: Failed`, `Task try 2: Failed`, `Task try 3: Success`.

2026-05-18, 22:45:15	run_spark_calculation	success	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:45:15	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:45:15	run_spark_calculation	running	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:45:15	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:45:03	run_spark_calculation	failed	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:45:03	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:45:03	run_spark_calculation	running	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:45:02	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:45:00	None	graph_data	None	Admin User	[{"dag_id", "taxi_big_data_pipeline"}]
2026-05-18, 22:44:51	run_spark_calculation	failed	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:44:51	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:44:51	run_spark_calculation	running	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:44:50	run_spark_calculation	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "run_spark_calculation", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:44:49	check_ingestion_status	success	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:44:49	check_ingestion_status	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "check_ingestion_status", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:44:49	check_ingestion_status	running	2026-05-18 22:44:46.466176+00:00	pvimakarova	None
2026-05-18, 22:44:48	check_ingestion_status	cli_task_run	None	airflow	["host_name": "b0375860f411", "full_command": "T:/home/airflow/local/bin/airflow", "tasks", "run", "taxi_big_data_pipeline", "check_ingestion_status", "manual__2026-05-18T22:44:46.466176+00:00", "--local", "--subdir", "DAGS_FOLDER/taxi_workflow.py"]
2026-05-18, 22:44:46	None	trigger	None	Admin User	[{"redirect_url", "/home"}, {"dag_id", "taxi_big_data_pipeline"}]

Рис. 3: Информация в логах

3 Spark: параллельная обработка данных и отказо-устойчивость

3.1 Постановка задачи

Используя Spark, необходимо реализовать параллельный расчет аналитического «Анти-рейтинга районов» города Нью-Йорк. Критерий анти-рейтинга: поиск ТОП-10 зон (pulocationid), имеющих наименьшую среднюю стоимость поездки (avg_amount), при условии, что из района было совершено более 5 поездок (trips_count > 5). Результат вычислений должен быть сохранен в таблицу default.taxi_anti_rating в ClickHouse.

pulocationid	trips_count	avg_amount
237	139093	20.51584731079143
141	66239	20.57955056688678
263	52034	20.597409193988486
151	19674	20.908890922029272
236	126458	20.94907866643379
193	1143	21.180419947506536
238	51104	21.30679281465207
239	72957	21.370517839275557
143	27284	21.445138909250822
142	89550	22.067786599664885

Рис. 4: Вывод результатов расчета анти-рейтинга в консоль

3.2 Отказоустойчивость

В рамках тестирования надежности распределенной системы был проведен эксперимент с принудительной имитацией аварии на узле. Изначально кластер находился в стабильном состоянии, все 3 Worker-узла имели статус **ALIVE**

Workers (3)

Worker id	Address	State	Cores	Memory	Resources
worker-20260519004548-172.19.0.5-33769	172.19.0.5:33769	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20260519004548-172.19.0.6-43477	172.19.0.6:43477	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20260519004548-172.19.0.7-45427	172.19.0.7:45427	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)	

Protonium Analytics (01)

Рис. 5: Состояние кластера до остановки worker'a: 3 узла ALIVE

На кластер была отправлена длительная ресурсоемкая задача. Прямо во время выполнения вычислений один из контейнеров (**spark-worker-2**) был жестко остановлен командой **docker-compose stop**.

Master-узел оперативно зафиксировал потерю Heartbeat-сигнала и перевел упавший узел в статус **DEAD** (Рис. 7). Благодаря фундаментальному механизму **RDD Lineage** (графу происхождения данных), Spark не прервал выполнение всего приложения. Менеджер кластера изолировал мертвый узел и автоматически перенаправил его незавершенные подзадачи на оставшиеся **spark-worker-1** и

spark-worker-3.

Workers (3)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260519004548-172.19.0.5-33769	172.19.0.5:33769	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)	
worker-20260519004548-172.19.0.6-43477	172.19.0.6:43477	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)	
worker-20260519004548-172.19.0.7-45427	172.19.0.7:45427	DEAD	1 (0 Used)	1024.0 MiB (0.0 B Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260519004746-0001	(null) PythonPi	2	1024.0 MiB		2026/05/19 00:47:46	spark	RUNNING	13 s

Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260519004634-0000	PythonPi	3	1024.0 MiB		2026/05/19 00:46:34	spark	FINISHED	32 s

Рис. 6: Выполнение работы с DEAD spark-worker-2

Workers (3)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260519004548-172.19.0.5-33769	172.19.0.5:33769	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20260519004548-172.19.0.6-43477	172.19.0.6:43477	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20260519004548-172.19.0.7-45427	172.19.0.7:45427	DEAD	1 (0 Used)	1024.0 MiB (0.0 B Used)	

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (2)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260519004746-0001	PythonPi	2	1024.0 MiB		2026/05/19 00:47:46	spark	FINISHED	5.9 min
app-20260519004634-0000	PythonPi	3	1024.0 MiB		2026/05/19 00:46:34	spark	FINISHED	32 s

Рис. 7: Успешное выполнение задачи с 2 spark-workers

3.3 Мониторинг и алертинг в Grafana

Для визуализации работы E-Commerce системы настроен интерактивный дашборд Grafana, подключенный напрямую к СУБД ClickHouse. На дашборд выведены следующие компоненты:



Рис. 8: Дэшборд в Grafana

Для контроля жизнеспособности пайплайна в Grafana настроено правило оповещения `No orders alert`. Запрос проверяет количество новых записей в таблице `taxi_trips` за последнюю минуту. Если значение падает ниже установленного порога (10 записей), алерт переходит в критическое состояние. В рамках тестирования Python-генератор был принудительно остановлен, в результате чего алерт успешно перешел в состояние **Firing** (загорелся красным цветом), сигнализируя об остановке стриминга.

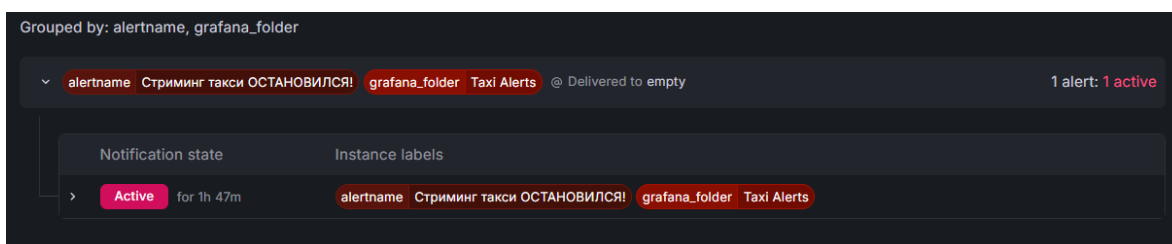


Рис. 9: Переход технического алерта в состояние Firing при остановке генератора

4 Заключение

В ходе выполнения курсового проекта был успешно спроектирован, отлажен и запущен полноценный пайплайн обработки больших данных. На практике проверена совместная работа оркестратора процессов (Airflow), высокопроизводительной аналитической базы данных (ClickHouse) и распределенного вычислительного движка (Spark).

Успешно реализованы механизмы автоматического восстановления задач при сбоях (Retry в Airflow), доказана отказоустойчивость архитектуры распределенных вычислений Spark и настроен сквозной мониторинг бизнес-метрик и технических метрик с системой экстренного оповещения (Alerting в Grafana).

Приложения

1. Официальный репозиторий открытых данных поездок такси Нью-Йорка (NYC TLC Trip Record Data): <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
2. Проект на GitHub: <https://github.com/pmakarova/big-data-practice>